

startTests.bat – Quick User Manual

startTests.bat – AGATE Kommandozeilen-Einstiegspunkt

`startTests.bat` ist der zentrale Kommandozeilen-Einstiegspunkt für den AGATE Server.

Über dasselbe Script können fünf typische Aufgaben ausgeführt werden:

1. Test Suite ausführen
2. Template + Testdaten instanziiieren
3. YAML validieren
4. Report analysieren
5. AGATE DSL anzeigen

1. Test Suite ausführen

Syntax

```
1 | startTests.bat <user> <instance> <app> <testSuite> [testCase]  
   | [priority]
```

Beispiel:

```
1 | startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
```

Bedeutung:

Parameter	Beispiel	Beschreibung
user	Testuser	Benutzer, unter dem der Test ausgeführt wird
instance	ACME	Testumgebung / Konfigurationsinstanz
app	TOSCATOAGATE	AGATE Application
testSuite	Instance_AGATE_Demo.yaml	Auszuführende Test Suite
testCase	optional	Nur einen bestimmten TestCase ausführen

priority	optional	Tests nach Priorität filtern
----------	----------	---------------------------------

Der Aufruf:

```
1 startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
```

bedeutet somit:

```
1 User      = Testuser
2 Instance  = ACME
3 Application = TOSCATOAGATE
4 Test Suite = Instance_AGATE_Demo.yaml
```

Die Test Suite wird nicht über einen hart codierten `data` -Pfad gesucht.

AGATE verwendet den in `project-agate.yaml` definierten `paths.tests` -Pfad und ergänzt automatisch Application und Dateinamen:

```
1 <paths.tests>\<app>\<testSuite>
```

Beispiel:

```
1 paths:
2   tests: 'C:\work\projects\agate-studio-new\agate-
server\ecs_fach\datax'
```

und:

```
1 startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
```

ergibt intern:

```
1 C:\work\projects\agate-studio-new\agate-
server\ecs_fach\datax\toscatoagate\Instance_AGATE_Demo.yaml
```

1.1. Einen bestimmten TestCase ausführen

Optional kann ein einzelner TestCase angegeben werden.

Beispiel:

```
1 startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
   AGATE_Demo
```

Damit wird nur:

```
1 AGATE_Demo
```

aus der angegebenen Test Suite ausgeführt.

1.2. TestCase und Priority angeben

Zusätzlich kann die Priorität angegeben werden:

```
1 startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml  
  AGATE_Demo HIGH
```

Damit kann die Testausführung zusätzlich auf die gewünschte Priorität eingeschränkt werden.

2. AGATE Project-Konfiguration

AGATE verwendet eine zentrale Projektdatei:

```
1 project-agate.yaml
```

Beispiel:

```
1 version: 1  
2  
3 project:  
4   name: fach  
5  
6 paths:  
7   tests: 'C:\work\projects\agate-studio-new\agate-  
server\ecs_fach\datax'  
8   responses: 'C:\work\projects\agate-studio-new\agate-  
server\ecs_fach\responses'  
9   reports: 'C:\work\projects\agate-studio-new\agate-  
server\ecs_fach\report'  
10  
11 config:  
12   environments: 'C:\work\projects\agate-studio-new\agate-  
server\ecs_fach\env\env.conf'  
13   users: 'C:\work\projects\agate-studio-new\agate-  
server\ecs_fach\env\users.conf'
```

Damit kennt AGATE zentral:

```
1 Tests  
2 Responses  
3 Reports  
4 Environment-Konfiguration  
5 User-Konfiguration
```

Die CLI-Aufrufe müssen diese Verzeichnisse deshalb nicht mehr vollständig angeben.

2.1. Konfigurierte Pfade

tests

Root-Verzeichnis aller AGATE Applications und Test Suites.

Beispiel:

```
1 C:\work\projects\agate-studio-new\agate-server\ecs_fach\datax
```

Darunter kann eine Application beispielsweise so aufgebaut sein:

```
1 | datax
2 |   └─ dmp11agate
3 |       └─ modules
4 |           └─ reusable
5 |               └─ template
6 |                   └─ CheckStatus.yaml
7 |                       └─ Instance_DMP SS12 V11 doAusschreibung.yaml
```

responses

Zentrales Verzeichnis für gespeicherte REST-/SOAP-Referenzantworten.

Beispiel:

```
1 | C:\work\projects\agate-studio-new\agate-server\ecs_fach\responses
```

reports

Root-Verzeichnis für erzeugte AGATE Reports.

Beispiel:

```
1 | C:\work\projects\agate-studio-new\agate-server\ecs_fach\report
```

environments

Environment-Konfiguration:

```
1 | C:\work\projects\agate-studio-new\agate-server\ecs_fach\env\env.conf
```

users

User-Konfiguration:

```
1 | C:\work\projects\agate-studio-new\agate-server\ecs_fach\env\users.conf
```

2.2. Relative und absolute Pfade

Die Pfade in `project-agate.yaml` können absolut oder relativ zum AGATE Project Root angegeben werden.

Beispiel absolut:

```
1 | paths:
2 |   tests: 'C:\work\projects\agate-studio-new\agate-server\ecs_fach\datax'
```

Beispiel relativ:

```
1 | paths:
```

```
2 tests: 'ecs_fach/datax'
```

2.3. Project Root

Wenn kein `--project` angegeben wird, verwendet AGATE das aktuelle Working Directory als Project Root.

Beispiel:

```
1 cd C:\work\projects\agate-studio-new\agate-server
```

danach:

```
1 startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
```

Optional kann ein Projekt explizit angegeben werden:

```
1 startTests.bat --project C:\TOSCA_PROJECTS\DMP11 Testuser ACME  
DMP11AGATE "Instance_DMP SS12 V11 doAusschreibung.yaml"
```

3. Konfiguration für die Testausführung

Die beim Start angegebenen Werte:

```
1 user  
2 instance
```

werden für die AGATE-Konfiguration verwendet.

Für:

```
1 startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
```

ist beispielsweise die Instance:

```
1 ACME
```

3.1. Environment-Konfiguration

Der Pfad zur Environment-Konfiguration wird über `project-agate.yaml` definiert.

Beispiel:

```
1 config:  
2   environments: 'C:\work\projects\agate-studio-new\agate-  
server\ecs_fach\env\env.conf'
```

In der Datei können Environment-Werte definiert werden.

Beispiel:

```
1 # --- TEST INSTANCE: ACME ---
```

```
2 ACME.math.url=http://api.mathjs.org
```

Im Test kann dieser Wert beispielsweise verwendet werden als:

```
1 endpoint: "{E[env.math.url]}"
```

3.2. User-Konfiguration

Auch der Pfad zur User-Konfiguration wird zentral über `project-agate.yaml` definiert.

Beispiel:

```
1 config:
2   users: 'C:\work\projects\agate-studio-new\agate-
server\ecs_fach\env\users.conf'
```

Für:

```
1 Instance = ACME
2 User     = Testuser
```

erwartet AGATE User-Einträge mit dem Prefix:

```
1 ACME.Testuser.
```

Beispiel:

```
1 # --- USER: ACME ---
2 ACME.Testuser.ip=127.0.0.1
3 ACME.Testuser.email=testuser@acme.local
```

Eine einzelne Zeile wie:

```
1 ACME.Testuser=Mustermann
```

reicht nicht aus, da AGATE nach Konfigurationseinträgen mit:

```
1 ACME.Testuser.
```

sucht.

4. Template instanziiieren

Die Instanziierung ist insbesondere für Template-basierte Tests wichtig, beispielsweise für Tests, die aus **Tosca TestCase Design** migriert wurden.

Dabei existieren zwei getrennte Artefakte:

```
1 Template YAML
2 +
3 CSV Testdaten
```

Beispiel:

```
1 AGATE_Demo.yaml
2 TCD_AGATE_Demo.csv
```

Diese beiden Dateien sind noch nicht die endgültige ausführbare Test Suite.

Mit `instantiate` werden Template und Testdaten kombiniert und daraus konkrete AGATE TestCases erzeugt.

4.1. Syntax

```
1 startTests.bat instantiate <appName> <templateFile> <dataFile>
```

Beispiel:

```
1 startTests.bat instantiate TOSCATOAGATE AGATE_Demo.yaml
  TCD_AGATE_Demo.csv
```

Dabei gilt:

Parameter	Beispiel	Bedeutung
appName	TOSCATOAGATE	Application, in deren <code>template</code> - Verzeichnis die Dateien liegen
templateFile	AGATE_Demo.yaml	YAML Template mit der Testlogik
dataFile	TCD_AGATE_Demo.csv	CSV mit den konkreten Testdaten

AGATE ermittelt die vollständigen Pfade automatisch über `project-agate.yaml`.

Intern werden daraus:

```
1 <paths.tests>\<appName>\template\<templateFile>
```

und:

```
1 <paths.tests>\<appName>\template\<dataFile>
```

Der Benutzer muss also keinen vollständigen Dateipfad mehr angeben.

4.2. Was passiert bei instantiate?

Das Template enthält die Testlogik, aber bestimmte Werte werden nicht direkt im YAML hinterlegt.

Beispiel:

```
1 - type: BUFFER
2   op: EXEC
3   name: 'x'
4   value: '{XL[x]}'
5
6 - type: BUFFER
7   op: EXEC
8   name: 'y'
9   value: '{XL[y]}'
10
11 - type: BUFFER
12   op: EXEC
13   name: 'z'
14   value: '{XL[z]}'
```

Die Ausdrücke:

```
1 {XL[x]}
2 {XL[y]}
3 {XL[z]}
```

verweisen auf Werte aus der CSV-Datei.

Beispielsweise kann die CSV unterschiedliche Datensätze enthalten:

```
1 x ; y ; z
2 2 ; 7 ; 3
3 22 ; 70 ; 3
```

Das Template definiert somit:

```
1 WAS getestet wird
```

und die CSV definiert:

```
1 MIT WELCHEN DATEN getestet wird
```

`instantiate` kombiniert beides.

4.3. Ablauf der Instanziierung

Der Ablauf kann vereinfacht so dargestellt werden:

```
1 AGATE_Demo.yaml
2   +
3   TCD_AGATE_Demo.csv
4     |
5     v
6 startTests.bat instantiate
7     |
8     v
9 Template wird gelesen
10    |
```

```

11      v
12 CSV-Datensätze werden gelesen
13      |
14      v
15 {XL[...] } werden mit konkreten Werten aufgelöst
16      |
17      v
18 konkrete TestCases werden erzeugt
19      |
20      v
21 ausführbare AGATE Test Suite

```

Das Template enthält beispielsweise:

```

1 name: 'x'
2 value: '{XL[x]}'

```

und der CSV-Datensatz enthält:

```

1 x = 2

```

Dann erhält der konkrete TestCase für diesen Datensatz:

```

1 x = 2

```

Beim nächsten CSV-Datensatz könnte dagegen gelten:

```

1 x = 22

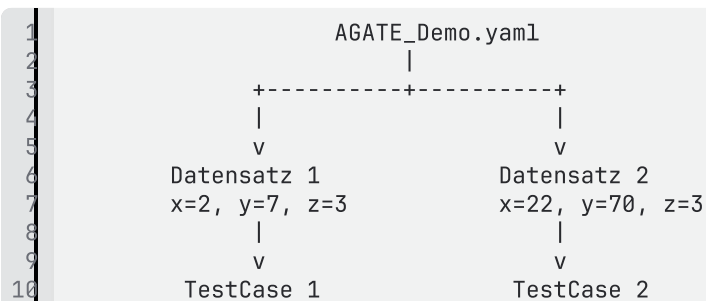
```

Die Testlogik bleibt identisch.

Nur die Testdaten ändern sich.

4.4. Warum gibt es Template und Instanz getrennt?

Der Vorteil ist, dass eine Testlogik nur einmal beschrieben werden muss.



Ohne Templates müsste für jeden Datensatz eine eigene YAML-Testdefinition gepflegt werden.

Mit Template + CSV kann dieselbe Testlogik für viele Datenkombinationen verwendet werden.

4.5. Application-Struktur

Die konkrete Verzeichnisstruktur wird über `paths.tests` bestimmt.

Beispiel:

```
1 <paths.tests>
2   └─ TOSCATOAGATE
3       ├── reusable
4       ├── modules
5       ├── template
6       │   ├── AGATE_Demo.yaml
7       │   └── TCD_AGATE_Demo.csv
8       └── Instance_AGATE_Demo.yaml
```

Die Template-Dateien werden instanziiert mit:

```
1 startTests.bat instantiate TOSCATOAGATE AGATE_Demo.yaml
  TCD_AGATE_Demo.csv
```

Das Ergebnis wird direkt im Application-Verzeichnis erzeugt:

```
1 <paths.tests>\TOSCATOAGATE\Instance_AGATE_Demo.yaml
```

4.6. Instanziierten Test ausführen

Nach erfolgreicher Instanziierung wird der erzeugte Test genau wie jede andere AGATE Test Suite ausgeführt.

Beispiel:

```
1 startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
```

Der vollständige Workflow ist:

```
1 AGATE_Demo.yaml
2   +
3   TCD_AGATE_Demo.csv
4     |
5     v
6   INSTANTIATE
7     |
8     v
9   Instance_AGATE_Demo.yaml
10    |
11    v
12  VALIDATE
13    |
14    v
15  EXECUTE
```

4.7. Empfohlener Workflow für Template-Tests

Schritt 1 – Template und CSV prüfen

```
1 <paths.tests>\TOSCATOAGATE\template\AGATE_Demo.yaml
2 <paths.tests>\TOSCATOAGATE\template\TCD_AGATE_Demo.csv
```

Schritt 2 – Instanzieren

```
1 startTests.bat instantiate TOSCATOAGATE AGATE_Demo.yaml  
TCD_AGATE_Demo.csv
```

Schritt 3 – Generiertes YAML validieren

```
1 startTests.bat validate TOSCATOAGATE Instance_AGATE_Demo.yaml
```

Schritt 4 – Test ausführen

```
1 startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
```

Damit ist der gesamte Template-Workflow:

```
1 Template  
2 +  
3 Testdaten  
4 |  
5 v  
6 INSTANTIATE  
7 |  
8 v  
9 konkrete Test Suite  
10 |  
11 v  
12 VALIDATE  
13 |  
14 v  
15 EXECUTE
```

5. YAML validieren

Mit `validate` kann eine AGATE YAML-Datei geprüft werden, ohne den eigentlichen Test auszuführen.

Syntax

```
1 startTests.bat validate <appName> <yamlFile>
```

Beispiel:

```
1 startTests.bat validate TOSCATOAGATE Instance_AGATE_Demo.yaml
```

Weiteres Beispiel:

```
1 startTests.bat validate MUHI CheckStatus.yaml
```

Bei Dateinamen mit Leerzeichen:

```
1 startTests.bat validate DMP11AGATE "Instance_DMP SS12 V11  
doAusschreibung.yaml"
```

Der vollständige Pfad muss nicht angegeben werden.

AGATE bildet ihn automatisch aus:

```
1 <paths.tests>\<appName>\<yamlFile>
```

Beispiel:

```
1 paths:
2   tests: 'C:\work\projects\agate-studio-new\agate-
server\ecs_fach\datax'
```

Aufruf:

```
1 startTests.bat validate DMP11AGATE "Instance_DMP SS12 V11
doAusschreibung.yaml"
```

ergibt intern:

```
1 C:\work\projects\agate-studio-new\agate-
server\ecs_fach\datax\dmp11agate\Instance_DMP SS12 V11
doAusschreibung.yaml
```

5.1. Erfolgreiche Validierung

Bei Erfolg erscheint:

```
1 =====
2   VALIDATION SUCCESSFUL
3   =====
```

Die Datei kann anschließend ausgeführt werden.

5.2. Fehlgeschlagene Validierung

Bei Fehlern erscheint beispielsweise:

```
1 =====
2   VALIDATION FAILED
3   =====
```

Zusätzlich liefert das Script einen Fehler-Exit-Code zurück.

Dies ist insbesondere auch für CI/CD-Pipelines nützlich.

5.3. Data-Flow-Validierung

Der Validator prüft unter anderem:

```
1 Buffer-Verwendung vor Initialisierung
2 Response-Verwendung vor Erzeugung
3 CALL / Reusable Datenfluss
4 LOOP Datenfluss
```

Beispiele für typische Fehler:

```
1 AGATE-V401
2 Variable/buffer is referenced before initialization
```

oder:

```
1 AGATE-V402
2 Response is used before it is created
```

5.4. LOOP-Semantik

Bei LOOP-Strukturen muss die tatsächliche Ausführungsreihenfolge berücksichtigt werden.

WHILE

```
1 condition
2 |
3 v
4 body
5 |
6 +----> condition
```

Die Condition wird also geprüft, bevor der Body erstmals ausgeführt wird.

Ein Buffer, der ausschließlich im Body erzeugt wird, darf deshalb nicht bereits in der ersten **WHILE**-Condition benötigt werden.

DO_WHILE

```
1 body
2 |
3 v
4 condition
5 |
6 +----> body
```

Beim **DO_WHILE** wird der Body dagegen mindestens einmal ausgeführt, bevor die Condition geprüft wird.

Damit ist beispielsweise folgende Konstruktion gültig:

```
1 - type: LOOP
2   mode: DO_WHILE
3   condition: "{B[patient_registered_count]} == 0"
4   maxIterations: 30
5   timeoutMs: 120000
6
7   steps:
8
9     - type: SQL
10      op: EXEC
11      command: >
12        SELECT COUNT(*)
13        FROM DMP_BETREUUNGSVERHAELTNIS
14      response: patient_registered_response
15
16     - type: SQL
17      op: BUFFER
18      response: patient_registered_response
19      row: "0"
20      column: "0"
21      name: patient_registered_count
```

Der Ablauf ist:

```
1 SQL EXEC
2 |
3 v
4 SQL BUFFER
5 |
6 v
7 patient_registered_count wird gesetzt
8 |
9 v
10 DO_WHILE Condition wird geprüft
```

`patient_registered_count` ist somit bei der Condition bereits vorhanden.

5.5. Empfehlung

Neue oder geänderte Tests sollten vor der Ausführung grundsätzlich validiert werden:

```
1 YAML
2 |
3 v
4 VALIDATE
5 |
6 +---- Fehler ----> korrigieren
7 |
8 v
9 EXECUTE
```

6. Report analysieren

Mit `analyze` kann ein bereits erzeugter AGATE JSON Report analysiert werden.

Syntax

```
1 startTests.bat analyze <jsonReport>
```

Beispiel:

```
1 startTests.bat analyze
  "ecs_fach\report\ECS_SYST_AUT1\dmp1lagate\Instance_DMP SS12 V11
  doAusschreibung\Latest_Report.json"
```

Der Analyzer:

```
1 JSON Report
2 |
3 v
4 JsonReportAnalyzer
5 |
6 v
7 AnalysisResult
8 |
9 +----> Console-Ausgabe
10 |
11 +----> Analysis JSON
```

Für:

```
1 Latest_Report.json
```

wird beispielsweise erzeugt:

```
1 Latest_Report_analysis.json
```

Die Analysis-Datei wird im selben Verzeichnis wie der ursprüngliche JSON Report erzeugt.

7. AGATE DSL anzeigen

Mit `describe` kann die eingebaute AGATE DSL-Dokumentation direkt über die Kommandozeile angezeigt werden.

Alle verfügbaren DSL-Typen

```
1 startTests.bat describe
```

Damit erhält man eine Übersicht der verfügbaren AGATE Step Types.

Einen bestimmten DSL-Typ anzeigen

REST:

```
1 startTests.bat describe REST
```

SOAP:

```
1 startTests.bat describe SOAP
```

Reusable / CALL:

```
1 startTests.bat describe CALL
```

Weitere Beispiele:

```
1 startTests.bat describe SQL
2 startTests.bat describe BUFFER
3 startTests.bat describe CMD
4 startTests.bat describe FILE
5 startTests.bat describe WAIT
```

`describe` ist damit die schnellste Referenz beim Schreiben eines AGATE Tests.

8. Typischer Workflow für einen normalen Test

Für eine bereits fertige Test Suite:

```
1 YAML
2 |
3 v
```

```
4  VALIDATE
5  |
6  v
7  EXECUTE
8  |
9  v
10 ANALYZE
```

Beispiel:

```
1 | startTests.bat validate TOSCATOAGATE Instance_AGATE_Demo.yaml
```

danach:

```
1 | startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
```

Optional anschließend:

```
1 | startTests.bat analyze "<reportPath>\Latest_Report.json"
```

9. Typischer Workflow für einen Template-Test

Bei Template-basierten Tests kommt vor der Validierung die Instanziierung:

```
1 | Template YAML
2 |   +
3 | CSV Testdaten
4 |   |
5 |   v
6 | INSTANTIATE
7 |   |
8 |   v
9 | konkrete Test Suite
10 |   |
11 |   v
12 | VALIDATE
13 |   |
14 |   v
15 | EXECUTE
16 |   |
17 |   v
18 | ANALYZE
```

Konkretes Beispiel:

```
1 | startTests.bat instantiate TOSCATOAGATE AGATE_Demo.yaml
   | TCD_AGATE_Demo.csv
```

danach:

```
1 | startTests.bat validate TOSCATOAGATE Instance_AGATE_Demo.yaml
```

anschließend:

```
1 | startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
```

und optional:

```
1 | startTests.bat analyze "<reportPath>\Latest_Report.json"
```

10. Quick Reference

Test ausführen

```
1 | startTests.bat <user> <instance> <app> <testSuite> [testCase]
   | [priority]
```

Beispiel:

```
1 | startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
```

Einzelnen TestCase ausführen

```
1 | startTests.bat Testuser ACME TOSCATOAGATE Instance_AGATE_Demo.yaml
   | AGATE_Demo
```

Template instanziiieren

```
1 | startTests.bat instantiate <appName> <templateFile> <dataFile>
```

Beispiel:

```
1 | startTests.bat instantiate TOSCATOAGATE AGATE_Demo.yaml
   | TCD_AGATE_Demo.csv
```

YAML validieren

```
1 | startTests.bat validate <appName> <yamlFile>
```

Beispiel:

```
1 | startTests.bat validate TOSCATOAGATE Instance_AGATE_Demo.yaml
```

oder:

```
1 | startTests.bat validate DMP11AGATE "Instance_DMP SS12 V11
   | doAusschreibung.yaml"
```

Report analysieren

```
1 | startTests.bat analyze <jsonReport>
```

Beispiel:

```
1 | startTests.bat analyze
   | "ecs_fach\report\ECS_SYST_AUT1\dmp11agate\Instance_DMP SS12 V11
   | doAusschreibung\Latest_Report.json"
```

DSL-Typen anzeigen

```
1 startTests.bat describe
```

oder:

```
1 startTests.bat describe REST
2 startTests.bat describe SOAP
3 startTests.bat describe CALL
4 startTests.bat describe SQL
```

11. Merksatz

```
1 project-agate.yaml
2   ↓
3   Wo liegen Tests, Responses, Reports und Config?
4
5   describe
6     ↓
7   Wie sieht die AGATE DSL aus?
8
9   instantiate
10    ↓
11  Template + CSV -> konkrete Tests
12
13  validate
14    ↓
15  Ist das YAML technisch korrekt?
16
17  startTests
18    ↓
19  Test wirklich ausführen
20
21  analyze
22    ↓
23  Was zeigt der erzeugte JSON Report?
```

Für Template-basierte Tests ist die wichtigste Reihenfolge:

```
1 INSTANTIATE -> VALIDATE -> EXECUTE -> ANALYZE
```

`instantiate` erzeugt aus einer wiederverwendbaren Testlogik und den dazugehörigen Testdaten eine konkrete Test Suite.

`validate` prüft die erzeugte Suite, ohne sie auszuführen.

Der normale Test Runner führt anschließend die konkrete Test Suite aus.

`analyze` kann danach den erzeugten JSON Report auswerten und zusätzlich eine `*_analysis.json` -Datei erzeugen.